Лабораторная работа 5

ДОПОЛНИТЕЛЬНЫЕ ИНДИВИДУАЛЬНЫЕ ЗАДАНИЯ

Разработайте пользовательские программы для решения указанных задач. Входные данные должны передаваться в программы через аргументы командной строки. Добавьте созданные вами программы в examples/Makefile. Имя каждой программы совпадает с именем файла исходного кода. Выполните сборку пользовательских программ. Поместите бинарные файлы созданных вами программ, а также программы для работы с файлами и программу shell на виртуальный диск, запустите pintos с оболочкой shell:

```
cd pintos/src/examples
make
cd ../userprog
make
cd build
pintos-mkdir fs.dsk --filesys-size=2
pintos --qemu --disk=fs.dsk \
  -p ../../examples/shell –a shell \
  -p ../../examples/rm –a rm \
  -p ../../examples/cat –a cat \
  -p ../../examples/cp –a cp \
  -p ../../examples/[myprog] –a [myprog] -- -f -q run shell
```

где [myprog] — имя вашей программы.

Подобным образом на диске можно разместить и другие файлы, например текстовые файлы, являющиеся входными данными для ваших программ.

После запуска pintos и запуска программы shell вы увидите приглашение командной строки pintos:

```
--
```

Программа shell считывает введенную команду и либо выполняет ее (если не указано exit), либо запускает программу, передавая ей аргументы командной строки.

Выполните несколько тестовых запусков вашей программы. Удалить ненужные файлы можно с помощью rm, просмотреть файл — cat, скопировать файл — с помощью cp.

Для разработки программ можно использовать минимальную библиотеку libc, предоставляемую как SDK Pintos. Заголовочные файлы библиотеки размещены в каталоге lib, подключать заголовочные файлы в коде следует через #include <…>

Варианты заданий:

1. Редактор (examples/editor.c)

Программа позволяет: создавать файлы для хранения строк, считывать, вставлять и удалять строки файла. Есть ограничения на количество и длину строк. Все параметры указываются через аргументы командной строки. При вставке строки стоящая на ее месте и следующие смещаются дальше (не заменяются). Программа должна поддерживать большие файлы (до 1Мб).

Справка: editor help

Создать файл: editor create [filename] [max_lines] [max_line_length]

Информация о файле (ограничения): editor info [filename]

Получить количество записанных строк: editor lines [filename]

Получить строку: editor read [filename] [line_number]

Удалить строку editor del [filename] [line_number]

Вставить строку на указанную позицию editor ins [filename] [line_number] [text]

2. Калькулятор (examples/calc.c)

Программа позволяет вычислять значения простейших арифметических выражений. При вычислении в истории сохраняется выражение и результат вычисления (хранится несколько последних запросов в файле). Историю можно просмотреть. Все параметры указываются через аргументы командной строки. Если файл с историей слишком мал для того, чтобы записать очередную строку, то его размер увеличивается как минимум на 1 кб. Выражение содержит знаки арифметических действий +-/%*, минус может быть унарным. Программа должна поддерживать большие файлы (до 1Мб).

Справка: calc help

Вычислить: calc eval [выражение]

Просмотреть историю вызовов: calc history

3. Переводчик (examples/translate.c)

В файле хранятся переводы нескольких английских слов на другой язык в виде "слово:перевод" — по одной записи на строке. Программа должна позволять перевести слово и добавить слово в словарь. Если слово уже есть в словаре — программа сообщает об этом и оно не добавляется. Если при добавлении слова размер файла не позволяет этого сделать, то файла словаря увеличивается как минимум на 1 кб. Программа должна поддерживать большие файлы (до 1Мб).

Справка: translate help

Перевести: translate find [слово]

Добавить в словарь: translate add [слово] [перевод]

4. Замена (examples/replace.c)

Программа позволяет заменить в текстовом файле все вхождения одной подстроки на другую. Результат записывается вместо исходного файла. При этом размер может измениться в большую или меньшую сторону. Допускается использовать временные файлы. Программа должна поддерживать большие файлы (до 1Мб).

Справка: replace help

Запуск: replace [filename] [искомая_подстрока] [на_что_заменить]

5. Множество (examples/digset.c)

Программа позволяет хранить в файле целые знаковые числа. Можно добавить число в файл, проверить наличие числа, удалить число. Данные в файле хранятся в отсортированном виде. Если число уже есть в файле — оно не добавляется. Если при попытке добавить число в файл его текущий размер не позволяет это сделать, то размер файла увеличивается как минимум на 1кб. Если файл не

существует — он должен быть автоматически создан. Программа должна поддерживать большие файлы (до 1Мб).

Справка: digset help

Добавить число: digset add [filename] [число]

Найти число: digset find [filename] [число]

Удалить число: digset del [filename] [число]

6. Журнал включений (examples/iplog.c)

Журнал подключений хранит IP-адрес компьютера и время его включения (некоторое число). Разработайте программу для работы с таким журналом. Журнал хранится в файле. При необходимости файл должен создаваться. Если при добавлении в файле недостаточно места — файл расширяется как минимум на 1 кб. Программа должна поддерживать большие файлы (до 1Мб).

Справка: iplog help

Добавить запись про компьютер: iplog add 192.168.76.5 4334

Найти все записи: iplog find 192.168.76.5

Проверка количества записей в журнале: iplog stat

При выводе записи выводятся в отсортированном по возрастанию времени виде.

7. Фоновые программы

Разработайте программу bgrun (examples/bgrun.c), которая позволяет запустить другую программу (указана через командную строку) и НЕ ДОЖИДАТЬСЯ ее завершения (т.е. запустить новую программу как фоновую задачу):

Запуск: bgrun daemon 1 2 3 — запустит программу daemon с аргументами командной строки 1 2 и 3 в фоне.

Для реализации необходимо считать значения из argv и выполнить системный вызов exec.

Программа loops (реализована в examples/loops.c) выполняет пустой цикл указанное в аргументе командной итераций и завершается.

Разработайте программу psinfo (examples/psinfo.c) — программа позволяет выводить на экран (в stdout) перечень запущенных в системе программ. Выводится tid (pid) программы, tid (pid) программы-создателя, имя исполняемого файла. Для реализации этой функциональности необходимо добавить реализацию системного вызова psinfo, копирующую информацию о запущенных процессахиз адресного пространства ядра в адресное пространство выполняющей системный вызов программы.

Протестируйте программу psinfo путем запуска нескольких экземпляров loops с помощью bgrun.

8. Системная информация

Разработайте программу examples/memstat.c. Программа позволяет вывести количество занятой и свободной оперативной памяти в системе. Информация может быть получена от ядра с помощью соответствующего системного вызова, который необходимо реализовать. Системный вызов должен скопировать информацию о количестве памяти в адресное пространство пользовательской программы (не выводить на экран), а вывод на экран должен быть осуществлён уже пользовательской программой. Протестируйте memstat путем запуска pintos на виртуальных компьютерах с разным объемом памяти.

Разработайте программу examples/fsinfo.c. Программа позволяет вывести в поток вывода информацию о текущей файловой системе: общее количество файлов на диске, занятый объем диска и свободный объем диска. Программа может получить эту информацию от ядра с помощью соответствующего системного вызова, который необходимо реализовать. Системный вызов должен скопировать информацию о диске в адресное пространство пользовательской программы (не

выводить на экран), а вывод на экран должен быть осуществлён уже пользовательской программой. Проверьте программу путем запуска при разных состояниях жесткого диска. Состояние можно изменять путем копирования и удаления файлов с помощью утилит cp rm.